From Single-Asset Timing to Portfolio Allocation

A Three-Phase Exploration of Backtesting Engines, Volatility Controls, and Modern Portfolio Theory

Zhixun (Ricardo) Zheng
ricardo05143018@gmail.com
GitHub Repository: https://github.com/ricardo05143018-creator/quant-backtest-engine

June 2026

Abstract

This project documents my independent evolution of a custom Python backtesting engine across three distinct phases of quantitative research. In Phase 1, I built a day-by-day simulation loop to eliminate the look-ahead bias common in vectorised online tutorials, but discovered that a fixed 5% stop-loss severely damaged performance on volatile assets like TSLA and BTC due to the "whipsaw" effect. In Phase 2, I attempted to fix this by introducing a volatility-adaptive Average True Range (ATR) stop-loss, only to fall into the trap of historical curve-fitting.

Realising that optimising parameters for individual assets is mathematically fragile, Phase 3 changes the focus completely from timing a single stock to spreading risk across a whole portfolio. By introducing Gold (GLD) as a non-correlated asset, I utilised Scipy to optimise for the Global Minimum Variance and Maximum Sharpe portfolios using Modern Portfolio Theory (MPT). Finally, acknowledging the underlying convex optimisation algorithm as a mathematical "black box" at my current level of study, I successfully verified the engine's outputs by brute-forcing a 10,000-portfolio Monte Carlo simulation to visually construct the Markowitz Efficient Frontier. Furthermore, to empirically prove the fragility of these mathematical models, I conducted an Out-of-Sample (OOS) stress test using 2026 data, demonstrating how historical optimization can act as an "error maximizer" vulnerable to market regime shifts.

---

1 Introduction and Motivation

I became interested in quantitative finance after noticing that many online trading tutorials use oversimplified backtests—typically just a few lines of pandas code that calculate signals across an entire dataset at once. This felt unrealistic to me: in real life, you cannot use tomorrow's price to make today's decision.

The goal of this project was to build something more careful, evolving through three distinct phases:

1. **Phase 1 (Engineering & Fixed Risk):** Construct a strict day-by-day backtesting engine to eliminate future data leakage, and observe how a fixed 5% stop-loss breaks down across different volatility regimes.
2. **Phase 2 (Adaptive Risk & Overfitting):** Implement a dynamic ATR (Average True Range) stop-loss to adapt to volatility, and test the robustness of grid-searched parameters via cross-asset validation.
3. **Phase 3 (Asset Allocation):** Abandon single-asset parameter tuning and implement Markowitz Modern Portfolio Theory (MPT) to lower risk mathematically through diversification.

What I discovered surprised me: solving one quantitative problem often introduces a more complex statistical trap. Understanding *why* these models fail became the most valuable part of the project.

2 Phase 1: Base Engine Architecture & The Fixed Stop-Loss Flaw

2.1 Eliminating Look-Ahead Bias

To prevent data leakage, I built the core trading logic as a day-by-day `for` loop. The constraint is absolute: **any decision on day t can only use data available at the close of day $t-1$.** Specifically, the engine employs a standard close-to-close execution assumption: trading signals are evaluated in the final minutes of day $t-1$ when the closing price is virtually established, executing the market order at a price near `closes[t-1]`. This allows the portfolio to be fully positioned to capture the continuous daily return of day t. While computationally slower than vectorised pandas operations, it is the only way to simulate a realistic trading environment.

2.2 The Whipsaw Effect on Volatile Assets

I tested a Dual Moving Average (DMA) strategy with a fixed 5% stop-loss across AAPL, TSLA, and BTC-USD using 2025 data. A grid search on AAPL found MA$(8, 60)$ to be optimal. On AAPL (lower volatility), the strategy entered once, held for 126 days without triggering the stop-loss, and returned +31.6% (Sharpe 2.16).

However, applying the exact same parameters to TSLA and Bitcoin revealed a serious structural flaw. On TSLA, the strategy entered four times, averaging just 17 days per trade, and returned -5.4% while buy-and-hold gained roughly 80%. On BTC-USD, two of three positions were stopped out within weeks.

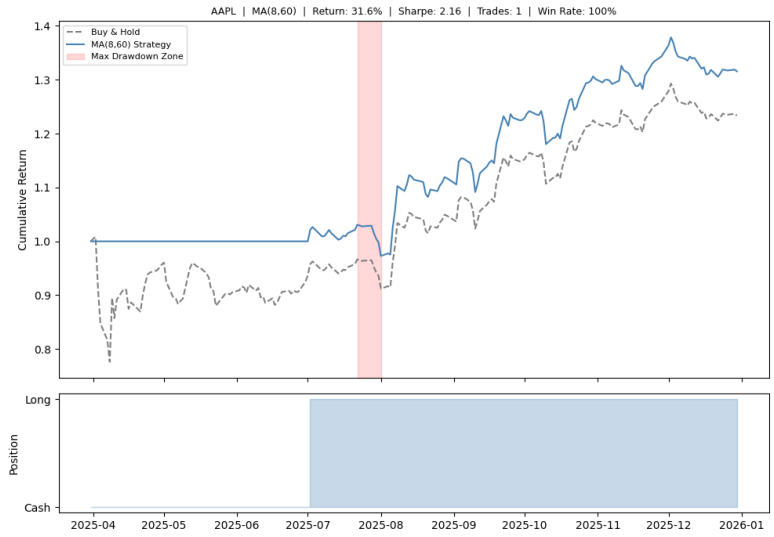

Figure 1: Phase 1: Benchmark performance on AAPL (2025). The low-volatility asset cleanly tracks the trend-following strategy without hitting the fixed 5% threshold.

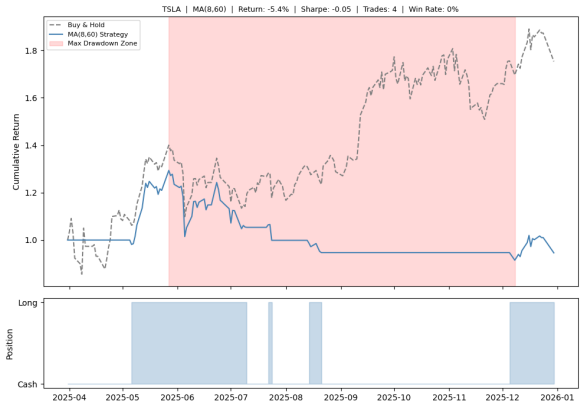

Figure 2: Phase 1: Whipsaw on TSLA.

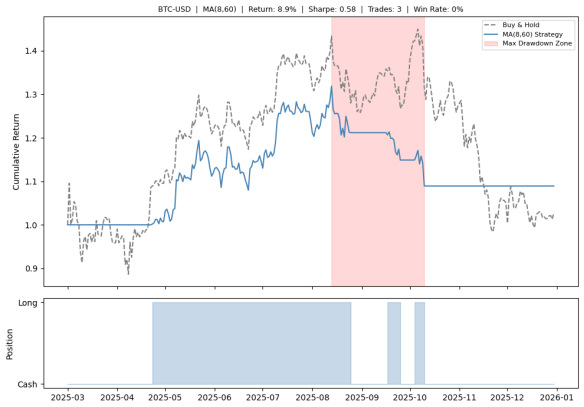

Figure 3: Phase 1: Whipsaw on BTC-USD.

The mechanism is straightforward. As shown in Figure 2 and Figure 3, day-to-day price swings in TSLA and BTC are large enough to regularly breach the 5% stop threshold even during upward trends. The strategy exits what it interprets as a large adverse move, but the price recovers—so the engine sells near a local low, only to re-enter higher on the next Golden Cross. Repeating this cycle steadily drains the portfolio. The fixed stop-loss implicitly assumes volatility is constant across assets, which is empirically false.

3 Phase 2: ATR Controls and The Overfitting Trap

To address the whipsaw effect, I implemented the **Average True Range (ATR)**, which calculates a rolling 14-day average of absolute price movements (including overnight gaps). I modified the engine to exit when the price drops below a dynamic cushion: Entry Price − (Multiplier × ATR).

I ran a 3D grid search on AAPL's data to find the optimal combination, which identified $1.5 \times$ ATR as the optimal threshold. It successfully prevented premature exits on AAPL during minor mid-year consolidations (Figure 4).

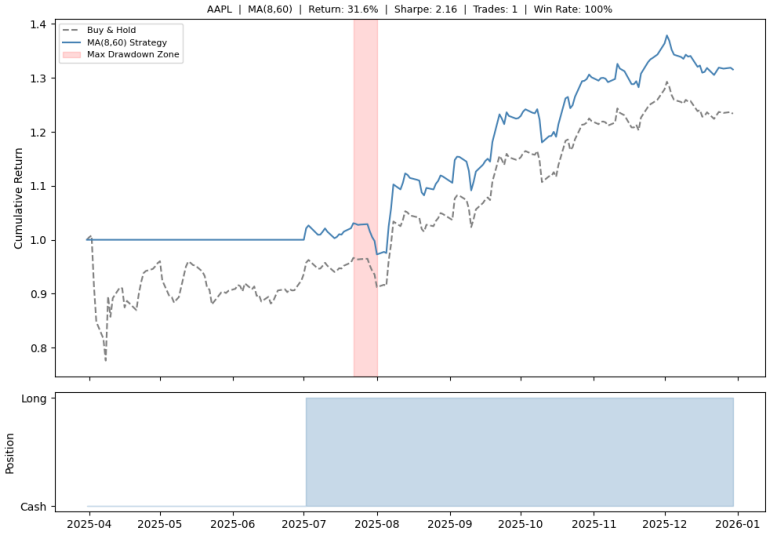

Figure 4: Phase 2: Optimised ATR stop-loss on AAPL (2025), yielding cleaner trend preservation.

3.1 The Illusion of Optimisation

However, when I conducted a cross-asset robustness test by applying this "optimised" 1.5x ATR parameter to TSLA and BTC-USD, the strategy catastrophically failed (Figure 5 and Figure 6).

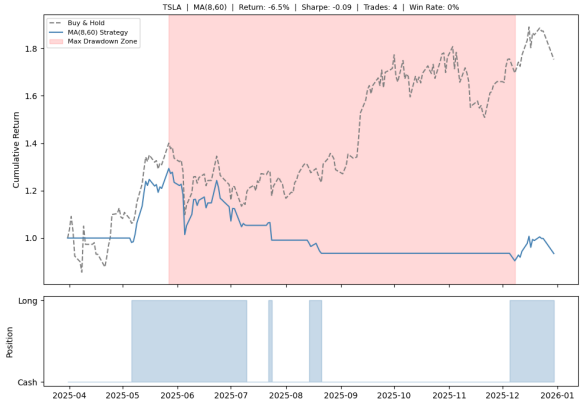

Figure 5: Phase 2: ATR failure on TSLA.

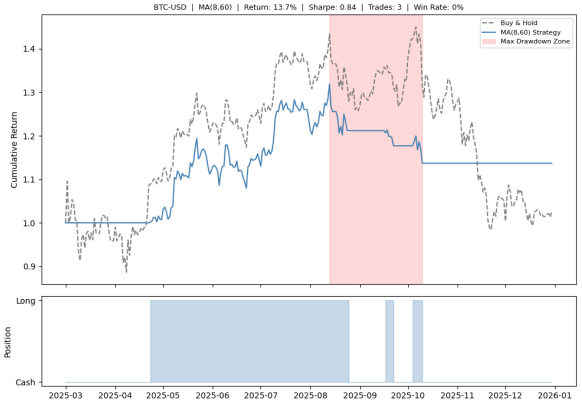

Figure 6: Phase 2: ATR failure on BTC.

This provided a brutal lesson in **historical curve-fitting**. The grid search algorithm simply memorised the precise, low-volatility structure of AAPL in 2025. When fed into TSLA's erratic price action or Bitcoin's macro gaps, this tight parameter triggered a relentless series of false exits. This proved that tuning stop-loss parameters asset-by-asset is a mathematically fragile exercise.

4 Phase 3: Modern Portfolio Theory (MPT) & Covariance

Phase 2 proved that looking for the perfect stop-loss parameters for just one stock is a dead end because the market changes too fast. To fundamentally reduce risk, I needed to step back from single-asset timing and move towards cross-asset allocation. I introduced a fourth asset, **SPDR Gold Shares (GLD)**, chosen specifically for its historical role as a non-correlated safe haven. Using my day-by-day engine data, I generated a daily returns matrix and calculated the 2025 correlation coefficients (Figure 7).

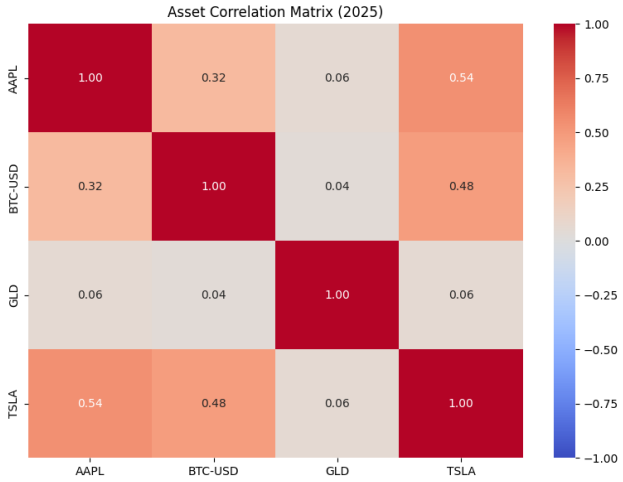

Figure 7: Phase 3: 2025 Asset Correlation Matrix. Note that GLD exhibits near-zero correlation with tech equity and crypto pools.

Because GLD moves independently of the other three highly correlated assets ($0.035 \sim 0.057$), combining them mathematically lowers the overall portfolio variance (σ_p^2) by neutralising the cross-covariance intersection terms in the Markowitz formula.

I utilised the `scipy.optimize` library to calculate two specific weight allocations:

- **Global Minimum Variance (GMV):** Allocated 72.9% to GLD, 16.0% to AAPL, 11.1% to BTC, and 0.0% to TSLA (Expected Return: 23.46%, Volatility: 15.50%).
- **Maximum Sharpe Ratio:** Allocated 87.1% to GLD, 12.9% to AAPL, and 0.0% to the rest (Expected Return: 28.14%, Volatility: 16.34%).

5 Brute-Force Verification via Monte Carlo Simulation

`SciPy` gives me the exact best weights, but the math behind its SLSQP solver involves multivariable calculus which is way above my current high school math level. Rather than blindly trusting a Python library as a "black box," I wrote a Monte Carlo simulation to verify its mathematical outputs. I programmed the engine to randomly generate 10,000 distinct portfolio weight combinations (ensuring $\sum w_i = 1$), calculate performance stats, and plot them.

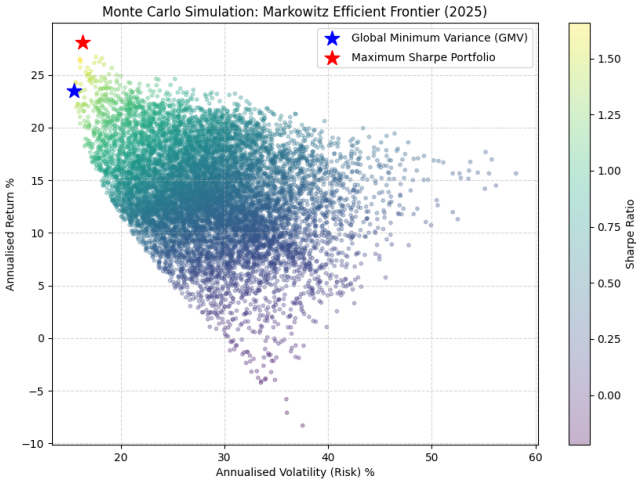

Figure 8: Monte Carlo Simulation of 10,000 random portfolios. The mathematical optimal solutions provided by Scipy perfectly land on the boundaries of the brute-force point cloud.

As shown in Figure 8, the random portfolios naturally form the classic Markowitz Efficient Frontier. The Scipy optimiser's GMV (Blue Star) and Max Sharpe (Red Star) land perfectly on the boundaries, visually proving the algorithm's accuracy.

5.1 Out-of-Sample Validation & The "Error Maximizer" Flaw

This simulation also exposed a notorious weakness of MPT. The Max Sharpe portfolio allocated a massive 87.1% to Gold. Because GLD experienced an unusually smooth and profitable bull run in 2025, the algorithm aggressively over-weighted it.

To empirically prove this vulnerability, I conducted an Out-of-Sample (OOS) test. I locked in the optimal Max Sharpe weights (fitted on 2025 data) and applied them to unseen market data from January to May 2026. The results were stark: the in-sample Sharpe ratio of 1.72 collapsed to an OOS Sharpe of just 0.41. While it barely outperformed a naive equal-weight ($1/N$) benchmark (Sharpe 0.22), this severe performance degradation confirmed my hypothesis. By feeding historical returns directly into the optimizer as expected returns, the model acts as an *error maximizer*. It perfectly curve-fits to the past but remains entirely blind to future regime shifts, representing yet another sophisticated form of look-ahead bias.

6 Conclusion and Academic Aspirations

The progression from Phase 1 to Phase 3 taught me that quantitative finance requires statistical thinking, not just coding ability. Look-ahead bias easily infects poorly structured code, single-asset parameter optimisation leads to fragile curve-fitting, and true risk management is achieved more effectively through asset diversification. Most importantly, Out-of-Sample testing proved that blind trust in mathematical optimisation algorithms without accounting for future regime shifts is fundamentally flawed.

Ultimately, using computational brute-force to visually map the Efficient Frontier allowed me to audit advanced mathematical tools honestly. These realisations directly motivate my ambition to study mathematics and statistics at the university level, where I aim to acquire the formal calculus and linear algebra tools necessary to handle market uncertainty rigorously.